
Oliver Krauss

Interfaces, Lambdas & Events

ALD | MTD.ba VZ SS26

Hagenberg 24.03.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Where We Left Off

Before we start -- let's connect to your last exercise.

In the last practical session you implemented **Circle**, **Rectangle**, and **Line** extending the abstract **Shape**. So far, a class can only inherit from *one* parent class -- but in real projects, objects often need multiple *capabilities* (multiple “roles”).

Recap: Where We Left Off

Before we start -- let's connect to your last exercise.

In the last practical session you implemented **Circle**, **Rectangle**, and **Line** extending the abstract **Shape**. So far, a class can only inherit from *one* parent class -- but in real projects, objects often need multiple *capabilities* (multiple “roles”).

Session 3 gave us:

- **Inheritance**: `Circle` extends `Shape` -- shared fields and methods
- **Abstract classes**: `Shape` defines a contract -- no direct instantiation
- **Polymorphism**: `shapes[i].draw(gc)` calls the right version at runtime
- Single inheritance: each class has **exactly one** parent class

Today: we show how interfaces let a class implement multiple capabilities, and how that idea shows up everywhere in JavaFX events.

The Single-Inheritance Problem

Java's rule: a class can extend **only one** class. This creates a problem:

```
1  // Fine -- Circle is a Shape:
2  class Circle extends Shape { ... }
3
4  // But what if Circle also needs to be:
5  // - Printable (can be printed to paper)
6  // - Saveable (can be saved to file)
7  // - Comparable (sortable by area)
8
9  // This is NOT valid Java:
10 class Circle extends Shape, Printable { }
11 // ERROR: cannot extend multiple classes
```

The limitation

A real **Circle** might play **many roles**:

- It **is a** Shape (has geometry)
- It **is** Printable
- It **is** Saveable
- It **is** Comparable

Single inheritance cannot express all of these.

Contracts Without Inheritance

Key Insight

Separate “**what an object is**” from “**what an object can do**”.

An **interface** is a **pure contract**:

- Defines **what** operations an object supports -- not *how* they work
- No implementation code (mostly), no fields, no state
- Any class can **implement** any number of interfaces
- A class **promises**: “I have all these methods”

Abstract class

Defines some implementation + some contracts. **What you are.**

Interface

Pure contract only. No fields, no state. **What you can do.**

Interface Syntax

```
1  public interface Drawable {
2      void draw(GraphicsContext gc); // implicitly public abstract
3  }
4
5  public interface Saveable {
6      void save(String filename);
7  }
8
9  // <T> is a type placeholder -- filled in by whoever uses the interface:
10 public interface Comparable<T> { // e.g. Comparable<Circle>: T = Circle
11     int compareTo(T other);      // then T here also means Circle
12 }
```

- Keyword **interface** instead of **class**
- Methods have **no body** -- they are implicitly **public abstract**
- Think of it as a **list of method signatures** -- a contract to fulfill
- **<T>**: a **type placeholder** -- we will look at generics later; for now, read **Comparable<Circle>** as “comparable to Circles”

The `implements` Keyword

A class fulfills an interface contract with `implements`:

```
1  public class Circle extends Shape implements Drawable {
2      // ...
3      @Override
4      public void draw(GraphicsContext gc) {
5          gc.setFill(this.getColor());
6          double d = this.radius * 2;
7          gc.fillOval(this.getX() - this.radius,
8                     this.getY() - this.radius, d, d);
9      }
10 }
```

-- `extends Shape`: Circle inherits from Shape (is-a Shape)

-- `implements Drawable`: Circle fulfills the Drawable contract

-- `@Override`: same as overriding a parent class method

-- If you claim `implements Drawable` but omit `draw()`: **compile error**

Misconception: “implements gives me code”

`implements` does **not** give you code -- it is a **promise** that you will provide it yourself.

```
1 public class Circle extends Shape implements Drawable {  
2     // forgot to implement draw(GraphicsContext gc)  
3 }
```

What happens?

Compile error: Circle is not abstract and does not override abstract method `draw(GraphicsContext)` in `Drawable`

Implementing Multiple Interfaces

A class can implement **as many interfaces as it needs**:

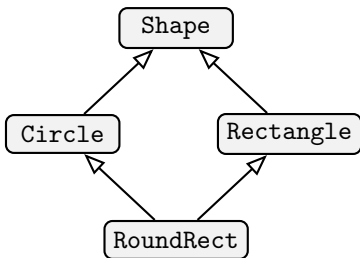
```
1  public class Circle extends Shape
2      implements Drawable, Saveable {
3
4      @Override
5      public void draw(GraphicsContext gc) { /* ... */ }
6
7      @Override
8      public void save(String filename) { /* ... */ }
9  }
```

The key advantage over multiple inheritance

Interfaces carry no **state** -- no fields.

Why interfaces avoid the diamond problem

The diamond problem (recap)



If both parents define the same method/field: which one is inherited?

Interfaces (Java's approach)

- **No fields** $\Rightarrow$ no shared state conflicts
- A class can implement **many roles**
- The compiler enforces the **contract** (all methods exist)

Your Turn: Compiles or Not?

Which of these snippets compile? Why or why not?

A)

```
1  interface Drawable {  
2      void draw(GraphicsContext gc);  
3  }  
4  class Circle implements Drawable {  
5      public void area() { ...}  
6  }
```

B)

```
1  interface Drawable {  
2      void draw(GraphicsContext gc);  
3  }  
4  class Circle implements Drawable {  
5      @Override  
6      public void draw(GraphicsContext gc) {  
7          // draws a circle  
8      }  
9  }
```

C)

```
1  interface Drawable {  
2      void draw(GraphicsContext gc);  
3  }  
4  Drawable d = new Drawable();
```

Your Turn: Compiles or Not? -- Answer

A) Compile error

`Circle` claims to **implement** `Drawable` but does not provide `draw()`. Java requires all abstract interface methods to be implemented. Fix: add the `draw()` method, or declare `Circle` **abstract**.

B) Compiles!

`Circle` fulfills the contract by providing `draw()` with the correct signature. `@Override` confirms the match.

C) Compile error

You cannot instantiate an interface with `new Drawable()` -- just like you cannot instantiate an abstract class. Fix: use `Drawable d = new Circle(...);`

Default Methods (Java 8+)

Since Java 8, interfaces can provide a **default implementation**:

```
1  public interface Drawable {  
2      void draw(GraphicsContext gc);    // abstract -- must implement  
3  
4      default void printInfo() {        // default -- inherited for free  
5          System.out.println("Drawable: " + this.toString());  
6      }  
7  }
```

- **default** methods have a body -- classes receive them **for free**
- Classes can still override default methods when needed
- **Why added?** Evolving APIs: adding a method to an existing interface would have broken all implementing classes -- default bodies provide backward compatibility

Default methods: diamond caveat

If a class implements **two interfaces** that both define a **default** method with the **same signature**, the compiler cannot choose one -- the class must resolve it.

```
1  interface A { default void log() { System.out.println("A"); } }
2  interface B { default void log() { System.out.println("B"); } }
3
4  class C implements A, B { }    // compile error: ambiguous
5
6  class C implements A, B {
7      @Override
8      public void log() { A.super.log(); }    // OK: explicit choice
9  }
```

Override `log()` and call `A.super.log()` or `B.super.log()` (or provide your own implementation). The compiler will tell you when this is required.

Interface vs. Abstract Class

Feature	Abstract Class	Interface
Has state (fields)?	Yes	No (only constants)
Has implementation?	Yes (concrete methods)	Default methods only
Constructor?	Yes	No
How many can a class use?	One (extends)	Many (implements)

Choose abstract class when...

- You need shared **state** (fields)
- You want shared **implementation**
- Strong is-a relationship

Choose interface when...

- You need **multiple types**
- Pure behavioral contract
- No shared state

Static Methods in Interfaces

Interfaces can also have **static utility methods**:

```
1  public interface Drawable {
2      void draw(GraphicsContext gc);
3
4      static Drawable createDefault() {
5          return new Circle(0, 0, 10, Color.BLACK);
6      }
7  }
8
9  // Called on the interface type, not on instances:
10 Drawable d = Drawable.createDefault();
```

- Static interface methods belong to the **interface itself**, not to implementing classes
- Useful for factory methods or helper utilities
- **circle.createDefault()** does **not** work -- must use **Drawable.createDefault()**

Interface Constants

Interfaces can define constants (implicitly `public static final`):

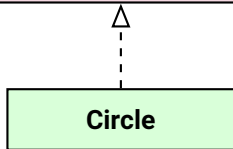
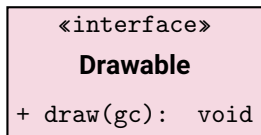
```
1  public interface ShapeConfig {
2      int MAX_SHAPES = 100;          // public static final
3      Color DEFAULT_COLOR = Color.BLACK;
4  }
5
6  class Drawing implements ShapeConfig {
7      private Shape[] shapes = new Shape[MAX_SHAPES];
8  }
```

Code smell -- Constant Interface Anti-Pattern

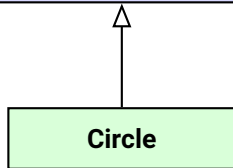
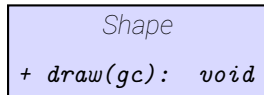
Using interfaces just to share constants is bad practice. If you only need constants, use a `class` with `static final` fields, or a Java `enum`. Interfaces are for **behavioral contracts**, not data.

UML for Interfaces

Interfaces use a different notation in UML:



implements (dashed)

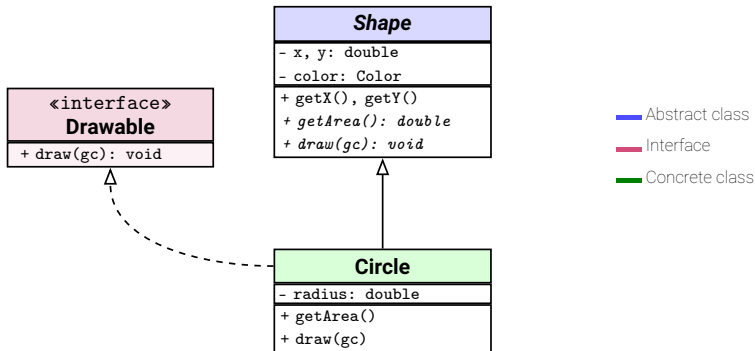


extends (solid)

Dashed arrow = realization (implements interface class)

Solid arrow = inheritance (extends class)

Our Shape Hierarchy with Interfaces



Solid arrow = inheritance Dashed arrow = realization (implements)

Programming to an Interface

Just like abstract class references, we can use **interface types** as variable types:

```
1  // Compile-time type: Drawable (the interface)
2  // Runtime type:      Circle   (the actual object)
3  Drawable d = new Circle(100, 100, 50, Color.RED);
4
5  d.draw(gc);           // OK: Drawable declares draw()
6  d.getRadius();        // COMPILE ERROR: Drawable has no getRadius()
```

- Any object that **implements** **Drawable** can be stored in a **Drawable** variable
- Same principle as **Shape s = new Circle(...)** -- but through an interface
- The compiler knows only the **interface type**; the runtime uses the **actual type**

Why “program to an interface”?

Code that depends only on **Drawable** (e.g. `void render(Drawable d)`) works with **any** implementation.

- Today: **Circle**, **Rectangle**, **Line**

- Tomorrow: new shapes you add later -- no change to **render()**

Takeaway

Design to the **interface**, not the concrete class. Flexibility and easier extension without touching existing code.

Polymorphic Arrays

Interfaces shine when grouping objects: an **array of the interface type** can hold any implementation:

```
1  Drawable[] items = new Drawable[3];
2  items[0] = new Circle(100, 100, 50, Color.RED);
3  items[1] = new Rectangle(200, 200, 80, 60, Color.BLUE);
4  items[2] = new Line(50, 50, 300, 300, Color.GREEN);
5
6  // Draw everything -- no instanceof, no casting:
7  for (Drawable d : items) {
8      d.draw(gc);
9  }
```

- **Drawable[]** accepts any object that implements **Drawable**
- The loop calls the **correct draw()** for each object at runtime (dynamic dispatch)
- Adding new shape types? The loop doesn't change

instanceof with Interfaces

`instanceof` works with interface types:

```
1  Shape shape = drawing.getShapes()[i];
2
3  // Check if this shape supports the Selectable interface:
4  if (shape instanceof Selectable s) {
5      s.select();          // s is automatically typed as Selectable
6  }
7
8  // Check for multiple interfaces:
9  if (shape instanceof Selectable s && shape instanceof Transformable t) {
10     s.select();
11     t.move(10, 0);
12 }
-- shape instanceof Selectable s: tests type and binds the cast variable in one step
-- Useful when you need interface-specific methods not available on the base type
-- Use sparingly -- if you find yourself checking types often, consider restructuring
```

Your Turn: Which Interfaces?

Design discussion with a neighbour

For the UE4 shapes -- **Ellipse**, **Triangle**, **Star**:

1. Which should implement **Drawable**? Why? Are there any that shouldn't?
2. Should **Line** implement a **Measurable** interface (e.g. `double getLength()`)? Why?
3. Imagine a **Selectable** interface with `select()`, `deselect()`, `isSelected()`. Which shapes make sense to be selectable?
4. Could a shape implement both **Measurable** AND **Selectable**? Is that valid Java?

There is no single correct answer -- think about what makes sense for a drawing application.

Which Interfaces? -- Design Answer

1. **Drawable**: All shapes qualify -- **Ellipse**, **Triangle**, **Star** all need to be rendered. A shape you cannot draw would be unusual in a drawing app.
2. **Line implements Measurable**: Valid Java! `getLength()` could return the line length (e.g. `Math.hypot(dx, dy)`). Whether it's *useful* depends on the app requirements.
3. **Selectable**: All shapes make sense -- you should be able to select and move any shape on the canvas.
4. **Measurable AND Selectable?** Yes -- perfectly valid Java. A class can implement as many interfaces as needed. The compiler verifies that **all** required methods are provided.

Key takeaway: Interfaces describe *roles*. A shape can play multiple roles simultaneously -- there is no conflict.

Recap -- Interfaces

- An **interface** is a pure contract: method signatures, no state, no constructors
- **implements**: a class fulfills the interface contract (all methods, or compile error)
- A class can implement **multiple** interfaces -- solves the multi-role problem
- Interfaces differ from abstract classes: no fields, no constructors
- **Default methods**: optional implementations added in Java 8+
- **Programming to an interface**: use `Drawable d = new Circle(...)`
- UML: «**interface**» box, **dashed** realization arrow (vs. solid inheritance)

Any questions before we see how interfaces enable lambda functions?

Nested (Inner) Classes

Sometimes a class is only useful **inside** another class:

```
1  class DrawingApp {  
2      // A helper type that belongs to DrawingApp:  
3      class Tool {  
4          void use() { System.out.println("using tool"); }  
5      }  
6  }
```

- A class declared inside another class is a **nested** (inner) class
- Used for small helper types that are closely tied to the outer class

Anonymous Classes

Sometimes you want a class **only once** (no name, no separate file):

```
1  interface Action { void run(); }
2
3  Action a = new Action() {
4      @Override public void run() {
5          System.out.println("do it!");
6      }
7  };
```

- This creates an object of an **unnamed** class that implements **Action**
- Useful, but often **verbose** when the method body is tiny

The Verbose Problem (Before Lambdas)

JavaFX event handling before lambdas often looked like this:

```
1  Button btn = new Button("Click me");
2
3  // All this boilerplate just to do one thing:
4  btn.setOnAction(new EventHandler<ActionEvent>() {
5      @Override
6      public void handle(ActionEvent event) {
7          System.out.println("Button clicked!");
8      }
9  });
```

- Lots of boilerplate for a single **println**
- You must: name the interface, annotate, name the method, name the parameter
- **What you actually want to express:** "when clicked, print"

There must be a better way

Goal

Express “when clicked, print” **without** all the class/method boilerplate.

Java 8 solution: lambda expressions

Lambda Syntax: Expression Form

A lambda is a **concise notation** for implementing a functional interface:

`(parameters) -> expression`

```
1 // Full anonymous class:
2 Runnable r1 = new Runnable() {
3     @Override public void run() { System.out.println("Hello!"); }
4 };
5
6 // Lambda (expression form -- single statement):
7 Runnable r2 = () -> System.out.println("Hello!");
```

-- `()` -- empty parameter list (`Runnable.run()` has no params)

-- `->` -- "goes to" / "maps to"

-- Right side: the expression that becomes the method body

Lambda Syntax: JavaFX Event Example

Same idea in JavaFX: **provide just the handler body**:

```
1  Button btn = new Button("Click me");  
2  
3  // EventHandler with one expression:  
4  btn.setOnAction(e -> System.out.println("Clicked!"));
```

- **e** is the event parameter (type is inferred)
- **e -> ...** means: "when the event happens, do this"

Lambda Syntax: Block Form

For more than one statement, use the **block form**:

```
(parameters) -> { statements; return value; }
```

```
1  // Block form with multiple statements (void return):  
2  btn.setOnAction(e -> {  
3      System.out.println("Button clicked!");  
4      counter++;  
5      redraw();  
6  });
```

- With `{...}`: need an explicit **return** for non-void methods
- Without `{...}`: single expression, value returned implicitly

Example: Runnable

Runnable -- a task with no inputs and no return value:

```
1  // Before lambdas (anonymous class):
2  Runnable r1 = new Runnable() {
3      @Override
4      public void run() { System.out.println("Hello from a thread!"); }
5  };
6
7  // With lambda:
8  Runnable r2 = () -> System.out.println("Hello from a thread!");
9
10 // Running them (same for both):
11 r1.run();
12 r2.run();
13
14 // Common JavaFX use -- schedule a UI update safely:
15 Platform.runLater(() -> label.setText("Data loaded!"));
```

Example: A Tiny “Function”

We can treat behavior like a value by storing it in a variable:

```
1  interface IntTransformer {
2      int apply(int x);
3  }
4
5  // A lambda is just an implementation of apply(...):
6  IntTransformer twice = x -> 2 * x;
7  IntTransformer plus1 = x -> x + 1;
8
9  System.out.println(twice.apply(10)); // 20
10 System.out.println(plus1.apply(10)); // 11
```

The lambda provides the body of **apply(int)** (a single-method interface).

From Anonymous Class to Lambda

Same behavior, less boilerplate:

```
1  // A) Runnable (before):
2  Runnable r1 = new Runnable() {
3      public void run() { counter++; }
4  };
5  // A) Runnable (after):
6  Runnable r2 = () -> counter++;
7
8  // B) JavaFX EventHandler (before):
9  btn.setOnAction(new EventHandler<ActionEvent>() {
10     public void handle(ActionEvent e) {
11         label.setText("Clicked: " + e.getSource());
12     }
13 });
14 // B) JavaFX EventHandler (after):
15 btn.setOnAction(e -> label.setText("Clicked: " + e.getSource()));
```

Lambdas Can Read Enclosing Variables

A lambda can **read** local variables from the scope where it is written:

```
1    double threshold = 100.0;
2
3    Runnable r = () -> System.out.println("Threshold is " + threshold);
4    r.run();    // prints: Threshold is 100.0
```

The lambda “captures” **threshold** when it is created. Instance fields (**this...**) can be read (and modified) in the same way.

Captured Variables Must Be “Effectively Final”

Local variables used inside a lambda must not be reassigned after the first assignment:

```
1  double threshold = 100.0;
2  threshold = 200.0;    // re-assignment
3
4  Runnable r = () -> System.out.println(threshold);
5  // COMPILER ERROR: Variable used in lambda should be effectively final
```

Why? The lambda might run *later* (e.g. on a button click), when the local variable no longer exists. Java captures its **value** and requires that value not change.

Method References: Static Methods

When a lambda just **delegates to an existing method**, use a method reference:

`ClassName::methodName`

```
1  interface StringToInt { int apply(String s); }
2  interface DoubleBinOp { double apply(double a, double b); }
3  interface StringPrinter { void print(String s); }
4
5  // Lambda:
6  StringToInt parser1 = s -> Integer.parseInt(s);
7  // Method reference (same thing, shorter):
8  StringToInt parser2 = Integer::parseInt;
9
10 // More examples:
11 DoubleBinOp max      = (a, b) -> Math.max(a, b);
12 DoubleBinOp maxRef   = Math::max;
```

Method References: Same Thing, Shorter

```
1 // Lambda:
2 StringPrinter printer = s -> System.out.println(s);
3 // Method reference (same thing):
4 StringPrinter printerRef = System.out::println;
```

Takeaway

Method references are just a **shorthand**: no new functionality, just clearer code when the lambda body is a single method call.

Bound instance (**obj::method**) and constructor (**Class::new**) references also exist -- see the backup slides at the end if you want to explore further.

Preview: Streams API

Lambdas + method references unlock the powerful **Streams API**:

```
1  List<Shape> shapes = drawing.getShapes();
2
3  // Find all large shapes and print their areas:
4  shapes.stream()
5      .filter(s -> s.getArea() > 100)
6      .forEach(s -> System.out.println(s.getArea()));
7
8  // Collect only circles into a new list:
9  List<Shape> circles = shapes.stream()
10     .filter(s -> s instanceof Circle)
11     .collect(Collectors.toList());
12
13  // Get all areas as a list:
14  List<Double> areas = shapes.stream()
15     .map(Shape::getArea)
16     .collect(Collectors.toList());
```

Recap -- Lambda Functions

- A **functional interface** has exactly one abstract method -- can be replaced by a lambda
- Lambda syntax: `(params) -> expression` or `(params) -> { block; }`
- **Type inference**: parameter types inferred from the target functional interface
- **Captured variables**: must be effectively final (local vars); instance fields are unrestricted
- **Method references**: `Class::method`, `obj::method`, `Class::new` -- shorthand for simple lambdas
- Lambdas = concise implementations of functional interfaces; same behavior, far less boilerplate

Any questions before we see lambdas in action with JavaFX events?

Event Model Overview

JavaFX uses an **event-driven model**:



- **Source**: the UI component (button, canvas, scene) the user interacts with
- **Event**: an object containing event data (position, key code, button, ...)
- **Handler**: your lambda implementing **EventHandler<T>** that reacts
- You register handlers with **setOn...** methods: **setOnMouseClicked**, **setOnKeyPressed**, etc.

setMouseClicked

Register a handler for mouse click events on a canvas:

```
1 Canvas canvas = new Canvas(800, 600);
2 canvas.setMouseClicked(e -> {
3     System.out.println("Clicked at: " + e.getX() + ", " + e.getY());
4 });
```

- `setMouseClicked` expects `EventHandler<MouseEvent>`
- `EventHandler<MouseEvent>` has exactly one method: `handle(MouseEvent e)`
- It is a **functional interface** -- so our lambda works directly
- `e` is the `MouseEvent` object with all event data

What Java sees

```
1 canvas.setMouseClicked(
2     new EventHandler<MouseEvent>() {
3         public void handle(
4             MouseEvent e) {
5             // your lambda body
6         }
7     });
```

MouseEvent API

Key methods on the **MouseEvent** object:

```
1  canvas.setOnMouseClicked(e -> {
2      double x = e.getX();           // x position in canvas
3      double y = e.getY();           // y position in canvas
4      MouseButton btn = e.getButton(); // which button was pressed
5      int clicks = e.getClickCount(); // 1 = single, 2 = double-click
6
7      if (btn == MouseButton.PRIMARY) {
8          addShapeAt(x, y);           // left click: create
9      } else if (btn == MouseButton.SECONDARY) {
10         removeShapeAt(x, y);         // right click: delete
11     }
12
13     if (clicks == 2) {
14         editShape(x, y);             // double-click: edit
15     }
16 });
```

Create a Shape at the Click Position

Combining everything: click the canvas to add a shape:

```
1  canvas.setOnMouseClicked(e -> {
2      if (e.getButton() == MouseButton.PRIMARY) {
3          Shape newShape = new Circle(
4              e.getX(), e.getY(), 30, Color.RED);
5          drawing.add(newShape);
6          redraw();
7      }
8  });
9
10 private void redraw() {
11     GraphicsContext gc = canvas.getGraphicsContext2D();
12     gc.clearRect(0, 0, canvas.getWidth(), canvas.getHeight());
13     drawing.drawAll(gc);
14 }
```

The core interaction loop: **event** → **update state** → **redraw**.

setOnKeyPressed

Register a handler for keyboard events -- **on the scene, not the canvas**:

```
1  @Override
2  public void start(Stage primaryStage) {
3      Scene scene = new Scene(root, 800, 600);
4
5      scene.setOnKeyPressed(e -> {
6          System.out.println("Key pressed: " + e.getCode());
7      });
8
9      // Canvas must request focus to receive key events
10     canvas.setFocusTraversable(true);
11     canvas.requestFocus();
12 }
```

Register on the Scene for global key handling

Key events go to the **focused element**. Register on the **Scene** so key presses work regardless of which control is focused, or on the **Canvas** after requesting focus.

KeyEvent and KeyCode

KeyCode is an enum with constants for every key:

```
1  scene.setOnKeyPressed(e -> {
2      switch (e.getCode()) {
3          case DELETE -> {
4              if (selectedShape != null) {
5                  drawing.remove(selectedShape);
6                  selectedShape = null;
7                  redraw();
8              }
9          }
10         case UP      -> moveSelected(0,  -5);
11         case DOWN    -> moveSelected(0,   5);
12         case LEFT    -> moveSelected(-5,  0);
13         case RIGHT   -> moveSelected( 5,  0);
14         case ESCAPE  -> deselectAll();
15     }
16 });
```


Bonus: `setOnMouseMoved` -- Hover Highlight

`setOnMouseMoved` fires as the mouse moves without a button held:

```
1  canvas.setOnMouseMoved(e -> {
2      // findAt() checks each shape and returns the topmost one at (x,y):
3      Shape hover = drawing.findAt(e.getX(), e.getY());
4      if (hover instanceof Selectable s) {
5          hoveredShape = hover;    // highlight shape under cursor
6      } else {
7          hoveredShape = null;
8      }
9      redraw(); // draw() checks hoveredShape for highlight
10 }
```

- `setOnMouseMoved`: mouse moves, **no** button pressed
- `setOnMouseDragged`: mouse moves **with** button held (for drag-to-resize)
- Hover highlighting is a UX touch that makes apps feel polished
- **Caution**: `redraw()` on every mouse move can be CPU-intensive

Focus and `requestFocus()`

Keyboard events only reach the **focused** node:

```
1  // Allow the canvas to receive keyboard focus:
2  canvas.setFocusTraversable(true);
3
4  // Request focus when the app starts:
5  canvas.requestFocus();
6
7  // Or: restore focus whenever the canvas is clicked:
8  canvas.setOnMouseClicked(e -> {
9      canvas.requestFocus(); // keyboard events come here after click
10     // ... rest of click handler
11 });
```

Common pitfall

You add **`setOnKeyPressed`** to the canvas, run the app, press a key -- nothing happens. The canvas has no focus. Fix: **`setFocusTraversable(true)`** + **`requestFocus()`**.

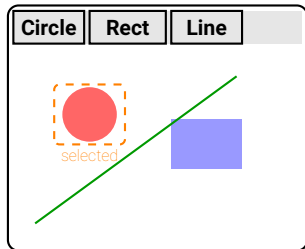
Goal: A Simple Vector Drawing Tool

What we are building:

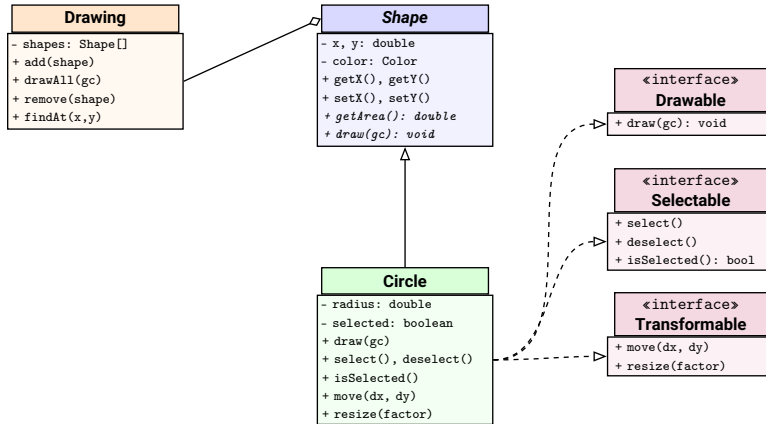
- A canvas where you draw shapes by clicking
- Toolbar buttons to select the shape type
- Click on an existing shape to select it
- Arrow keys to move the selected shape
- Delete key to remove the selected shape

What it demonstrates:

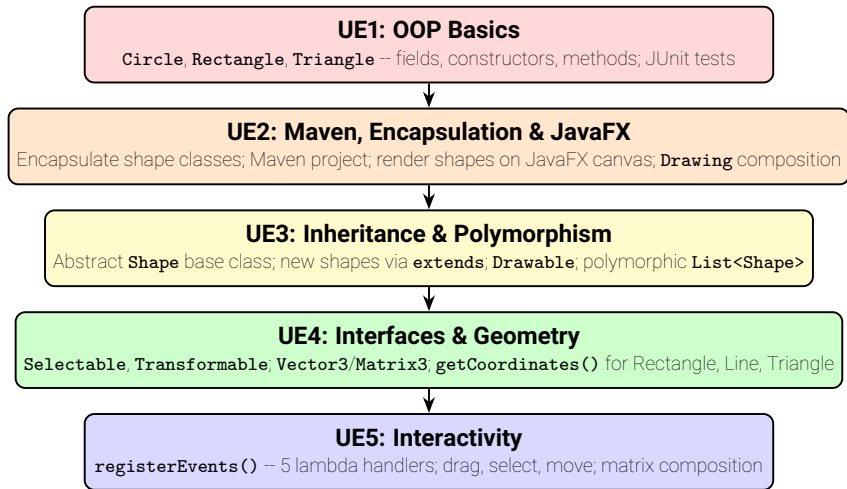
- All four sessions of the OOP/JavaFX arc working together
- Interfaces, lambdas, events, and polymorphism in a real app



Full Architecture Diagram



Sessions 1--5: The Practical Arc



You are here! UE5
puts this into practice.

UI Layout: BorderPane + HBox

```
1  @Override
2  public void start(Stage primaryStage) {
3      canvas = new Canvas(800, 550);
4
5      // Toolbar with shape-type buttons:
6      Button btnCircle = new Button("Circle");
7      Button btnRect   = new Button("Rectangle");
8      Button btnLine   = new Button("Line");
9      HBox toolbar = new HBox(10, btnCircle, btnRect, btnLine);
10     toolbar.setPadding(new Insets(8));
11     // BorderPane: toolbar on top, canvas in center:
12     BorderPane root = new BorderPane();
13     root.setTop(toolbar);
14     root.setCenter(canvas);
15
16     Scene scene = new Scene(root, 800, 600);
17     ...
18 }
```

Application State

The app needs to track what type of shape to create next, and which shape is selected:

```
1  public class ShapeApp extends Application {  
2  
3      private Canvas  canvas;  
4      private Drawing drawing = new Drawing(100);  
5  
6      // Enum for the current tool mode:  
7      private enum ShapeType { CIRCLE, RECTANGLE, LINE }  
8      private ShapeType currentType = ShapeType.CIRCLE;  
9  
10     // Currently selected shape (null if nothing selected):  
11     private Shape selectedShape = null;  
12  
13     // ...  
14 }
```

Button → State Change

Connect toolbar buttons to the application state with lambdas:

```
1  Button btnCircle = new Button("Circle");
2  Button btnRect   = new Button("Rectangle");
3  Button btnLine   = new Button("Line");
4
5  // Lambda: clicking a button updates the current tool mode:
6  btnCircle.setOnAction(e -> currentType = ShapeType.CIRCLE);
7  btnRect   .setOnAction(e -> currentType = ShapeType.RECTANGLE);
8  btnLine   .setOnAction(e -> currentType = ShapeType.LINE);
```

- **setOnAction** takes **EventHandler<ActionEvent>** -- a functional interface
- Each lambda captures **currentType** (an instance field -- modification allowed)
- No anonymous classes, no separate listener classes -- three clean lambdas

Click → Create Shape

The canvas click handler creates the right shape based on the current tool:

```
1  canvas.setOnMouseClicked(e -> {
2      canvas.requestFocus();
3      if (e.getButton() == MouseButton.PRIMARY) {
4          Shape newShape = switch (currentType) {
5              case CIRCLE    -> new Circle(e.getX(), e.getY(),
6                                          30, Color.RED);
7              case RECTANGLE -> new Rectangle(e.getX(), e.getY(),
8                                              80, 50, Color.BLUE);
9              case LINE      -> new Line(e.getX(), e.getY(),
10                                         e.getX() + 80,
11                                         e.getY() + 40, Color.GREEN);
12          };
13          drawing.add(newShape);
14          redraw();
15      }
16  });
```

The **Selectable** Interface

Define the contract for shapes that can be selected:

```
1  public interface Selectable {
2      /** Mark this shape as selected. */
3      void select();
4
5      /** Deselect this shape. */
6      void deselect();
7
8      /** Returns true if this shape is currently selected. */
9      boolean isSelected();
10 }
```

- Pure contract -- no implementation, no state
- Any shape that wants to be selectable implements this
- Code that needs to select/deselect works with **Selectable** references -- no **instanceof**
- Useful for: drawing selection highlights, enabling toolbar actions

The Transformable Interface

Define the contract for shapes that can be moved and resized:

```
1  public interface Transformable {
2      /** Move by dx pixels horizontally, dy vertically. */
3      void move(double dx, double dy);
4
5      /** Resize by a scale factor (2.0 = double, 0.5 = half). */
6      void resize(double factor);
7  }
```

Note on `move()`: where are the x, y fields?

Shape stores `x` and `y` as **private** fields. `move()` needs to change them. **Recommended**: add **protected void setX(double x)** and **protected void setY(double y)** to **Shape** -- accessible in subclasses, not exposed to external code.

Circle Implements Interfaces I

```
1  public class Circle extends Shape implements
2      Drawable, Selectable, Transformable {
3
4      private boolean selected = false;
5      // --- Selectable ---
6      @Override public void select()      { this.selected = true; }
7      @Override public void deselect()    { this.selected = false; }
8      @Override public boolean isSelected() { return this.selected; }
9
10     // --- Transformable ---
11     @Override public void move(double dx, double dy) {
12         setX(getX() + dx);    // Shape provides protected setX/setY
13         setY(getY() + dy);
14     }
15     @Override public void resize(double factor) {
16         this.radius *= factor;
17     }
```

Circle Implements Interfaces II

```
1      ...
2
3      // --- Drawable: highlight selection ring when selected ---
4      @Override public void draw(GraphicsContext gc) {
5          gc.setFill(getColor());
6          gc.fillOval(getX()-radius, getY()-radius, radius*2, radius*2);
7          if (selected) {
8              gc.setStroke(Color.ORANGE); gc.setLineWidth(3);
9              gc.strokeOval(getX()-radius-3, getY()-radius-3,
10                           (radius+3)*2, (radius+3)*2);
11          }
12      }
13  }
```

Click-to-Select Logic

Find the shape at the click position and select it:

```
1  canvas.setOnMouseClicked(e -> {
2      canvas.requestFocus();
3      Shape found = drawing.findAt(e.getX(), e.getY());
4      if (found instanceof Selectable s) {
5          // Deselect previous:
6          if (selectedShape instanceof Selectable prev) prev.deselect();
7          selectedShape = found;
8          s.select();
9      } else if (e.getButton() == MouseButton.PRIMARY) {
10         // No shape at this position -- create a new one:
11         if (selectedShape instanceof Selectable prev) prev.deselect();
12         selectedShape = null;
13         // ... create shape as in the earlier slide
14     }
15     redraw();
16 });
```

`drawing.findAt(x, y)` searches shapes in reverse (topmost first) and returns the first.

Arrow Keys → Move Selected Shape

```
1  scene.setOnKeyPressed(e -> {
2      // Pattern matching: only proceed if selectedShape is Transformable:
3      if (!(selectedShape instanceof Transformable t)) return;
4      switch (e.getCode()) {
5          case UP      -> t.move(0, -5);
6          case DOWN    -> t.move(0,  5);
7          case LEFT     -> t.move(-5, 0);
8          case RIGHT    -> t.move( 5, 0);
9          case PLUS     -> t.resize(1.1); // grow 10%
10         case MINUS    -> t.resize(0.9); // shrink 10%
11         default       -> { return; }    // no-op keys: skip redraw
12     }
13     redraw();
14 });
```

Delete Key → Remove Shape

```
1  scene.setOnKeyPressed(e -> {
2      switch (e.getCode()) {
3          case DELETE, BACK_SPACE -> {
4              if (selectedShape != null) {
5                  drawing.remove(selectedShape);
6                  selectedShape = null;
7                  redraw();
8              }
9          }
10         case UP      -> { /* ... */ }
11         case DOWN    -> { /* ... */ }
12         // ...
13     }
14 });
```

- Both **DELETE** and **BACK_SPACE** trigger removal (standard UX on all platforms)
- **drawing.remove(selectedShape)**: removes from the shape collection
- Set **selectedShape = null** to avoid dangling references

Quick Retrieval: Test Yourself

Before looking at the summary -- can you answer these from memory?

1. What is the difference between an interface and an abstract class?
2. What is a functional interface, and why can it be replaced by a lambda?
3. You write `canvas.setOnMouseClicked(e -> ...)` -- what type is `e`?
4. A class `extends Shape implements Drawable, Selectable` -- is this valid Java? How many abstract methods must it implement?
5. To receive keyboard events on a `Canvas`, what two calls are needed?

Take some time to think, then we go through the answers.

Quick Retrieval: Answers

1. **Interface vs. abstract class:** Interface = pure contract (no fields, no state, no constructor), a class can implement many. Abstract class = partial implementation with fields, a class can extend only one.
2. **Functional interface + lambda:** Exactly one abstract method means Java can unambiguously map a lambda to that method -- no ambiguity about which method is being implemented.
3. **Type of e:** `MouseEvent` -- the event object delivered by JavaFX containing position, button, click count, etc.
4. **Valid?** Yes. Must implement: `draw()` from `Drawable`, plus `select()`, `deselect()`, `isSelected()` from `Selectable`, plus any abstract methods from `Shape` (`getArea()`, `draw()`).
5. **Keyboard on Canvas:** `canvas.setFocusTraversable(true)` and `canvas.requestFocus()`.

Summary

- **Interface**: pure contract -- method signatures only, no state, no constructors
- A class can **implement multiple** interfaces -- solves the multi-role problem
- **Default methods**: optional implementations in interfaces (Java 8+)
- **Functional interface**: exactly one abstract method -- can be replaced by a **lambda**
- Lambda syntax: `(params) -> expression` or `(params) -> { block; }`
- **Method references**: `Class::method`, `obj::method`, `Class::new`
- **JavaFX event model**: source $\rightarrow$ event object $\rightarrow$ handler (lambda)
- `setOnMouseClicked`, `setOnKeyPressed`: register event handlers with lambdas
- **Selectable + Transformable**: interfaces bring the interactive drawing app to life
- Sessions 1--4 complete: full OOP foundation + JavaFX event-driven programming

Backup: Functional Interfaces

The key insight: `EventHandler<ActionEvent>` has **exactly one** abstract method (`handle`).

Functional Interface

An interface with **exactly one abstract method** is called a **functional interface**. It can be replaced by a **lambda expression**.

```
1  // These are functional interfaces (1 abstract method each):
2  interface EventHandler<T> { void handle(T event); }
3  interface Runnable      { void run(); }
4  interface Comparator<T> { int compare(T a, T b); } // *
5
6  // This is NOT a functional interface (2 abstract methods):
7  interface Renderable {
8      void draw(GraphicsContext gc);
9      double getArea();
10 }
```

Backup: @FunctionalInterface

You can mark your own functional interfaces with `@FunctionalInterface`:

```
1  @FunctionalInterface
2  public interface Transformer {
3      Shape transform(Shape s);
4  }
5
6  // Adding a second abstract method causes a compile error:
7  @FunctionalInterface
8  public interface Broken {
9      void doSomething();
10     void doSomethingElse(); // ERROR: not a functional interface
11 }
```

- `@FunctionalInterface` is optional -- it does not change runtime behavior
- It documents **intent**: “this interface is designed for lambdas”
- The compiler checks: if you accidentally add a second method, you get a helpful error

Backup: java.util.function types

Java provides ready-made functional interfaces in `java.util.function`:

Interface	Method	Usage
<code>Predicate<T></code>	<code>boolean test(T t)</code>	filter, condition check
<code>Function<T,R></code>	<code>R apply(T t)</code>	transform a value
<code>Consumer<T></code>	<code>void accept(T t)</code>	process without result
<code>Supplier<T></code>	<code>T get()</code>	produce a value
<code>BiFunction<T,U,R></code>	<code>R apply(T t, U u)</code>	transform two values

```
1  Predicate<Shape>      isLarge = s -> s.getArea() > 500;
2  Function<Shape, String> desc    = s -> s.getClass().getSimpleName();
3  Consumer<Shape>      printer = s -> System.out.println(desc.apply(s));
4  Supplier<Shape>      factory = () -> new Circle(0, 0, 10, Color.BLACK);
```

Backup: Type inference in lambdas

Lambda parameter types are usually **inferred from context**:

```
1  // With explicit types (verbose but clear):
2  Comparator<Circle> c1 = (Circle a, Circle b) ->
3      Double.compare(a.getRadius(), b.getRadius());
4
5  // With inferred types (common style):
6  Comparator<Circle> c2 = (a, b) ->
7      Double.compare(a.getRadius(), b.getRadius());
8
9  // Single parameter: parentheses optional:
10 Consumer<Circle> printer = c -> System.out.println(c.getArea());
```

- Java infers **a** and **b** are **Circle** from **Comparator<Circle>**
- Single-parameter lambdas can omit parentheses: **e -> e.getX()**
- The compiler still checks everything (type safety)

Backup: Method References -- Instance Methods

Two kinds of instance method references:

```
1 // 1) Bound -- on a specific object:
2 Circle myCircle = new Circle(100, 100, 50, Color.RED);
3 Supplier<Double> getMyArea = myCircle::getArea;
4 // equivalent to: () -> myCircle.getArea()
5
6 // 2) Unbound -- on whatever instance is passed in:
7 Function<Circle, Double> getArea = Circle::getArea;
8 // equivalent to: (circle) -> circle.getArea()
9
10 // Unbound in a sort:
11 circles.sort(Comparator.comparing(Circle::getRadius));
```

-- **Bound:** method on a *specific* object (`myCircle::getArea`)

-- **Unbound:** method on *whatever instance is passed in* (`Circle::getArea`)

Backup: Method References -- Constructors

Reference a constructor with `ClassName::new`:

```
1 // Lambda:
2 Supplier<Drawing> factory1 = () -> new Drawing(100);
3 // Constructor reference (same thing):
4 Supplier<Drawing> factory2 = Drawing::new;
5
6 // With a Function (1 param -> 1 result):
7 Function<Integer, Drawing> creatorRef = Drawing::new;
8
9 // Usage:
10 Drawing d = factory2.get();
11 Drawing d2 = creatorRef.apply(50);
```

-- `Drawing::new` creates a `Drawing` when called

-- Constructor parameter types must match the functional interface's method signature